



DEPLY MANUAL



Werable
NAVICORE

1. Arquitectura general

La app de Wear OS es un cliente que consume el backend REST de TraveXperience (Node.js + Express) bajo el prefijo /api/v1. No incluye lógica de negocio propia: itinerarios, lugares cercanos y clima se calculan en el servidor.

Para que el reloj funcione de extremo a extremo se necesitan tres piezas desplegadas y comunicándose entre sí:

- Backend (Node.js + MySQL + MongoDB) — expone la API que el reloj consume.
- App móvil o Web — desde donde el usuario genera el código de 6 dígitos para vincular el reloj.
- App de Wear OS (este proyecto) — instalada en el reloj, configurada para apuntar a la URL del backend.

Nota: Este manual asume que ya tienes ambos proyectos (backend y wear) descomprimidos localmente. El Manual de Build cubre cómo compilar específicamente el cliente Wear OS con Android Studio/Gradle.

2. Requisitos del entorno

Requisito	Uso
Node.js	>= 18.0.0 (definido en package.json → engines)
MySQL	Motor relacional para Usuarios, Perfiles, Itinerarios
MongoDB	Motor documental para Reseñas, logs de actividad y metadatos de IA
npm	Incluido con Node.js, usado para instalar dependencias
Cuenta OpenWeatherMap	API Key gratuita para el módulo de clima
Cuenta Stripe	Claves de test o producción para el módulo de pagos
Google Cloud Console	Client ID (Google Sign-In) y API Key de Geocoding (Google Maps)

3. Despliegue del backend

3.1 Variables de entorno

El backend usa un archivo .env en la raíz del proyecto (nunca se sube al control de versiones). Debe contener estas variables:

Variable	Descripción
NODE_ENV	development / production
PORT	Puerto HTTP del servidor (por defecto 4000)

API_PUBLIC_URL	URL pública del backend, usada para armar links de fotos accesibles desde el celular
MYSQL_HOST / PORT / DATABASE / USER / PASSWORD	Credenciales de conexión a MySQL
MONGO_URI	Cadena de conexión completa a MongoDB
JWT_SECRET	Secreto para firmar los tokens de sesión (access + refresh)
JWT_EXPIRES_IN / JWT_REFRESH_EXPIRES_IN	Vigencia del accessToken y refreshToken
BCRYPT_SALT_ROUNDS	Costo del hash de contraseñas
GOOGLE_CLIENT_ID	Mismo Client ID usado en el frontend para “Continuar con Google”
AI_MODEL_CLUSTERING_PATH / AI_MODEL_BUDGET_PATH	Rutas a los modelos TensorFlow.js exportados (model.json + weights.bin)
WEARABLE_ACTIVE_WINDOW_MINUTES	Minutos de ventana para considerar un itinerario “activo” en el reloj
OPENWEATHER_API_KEY	API Key de OpenWeatherMap
STRIPE_SECRET_KEY / STRIPE_WEBHOOK_SECRET	Credenciales de Stripe
GOOGLE_MAPS_API_KEY	API Key de Geocoding (Google Maps Platform)

Importante: Nunca compartas ni subas a un repositorio público un archivo .env con valores reales. Usa siempre un .env.example con los mismos nombres de variable y valores de ejemplo (placeholders).

3.2 Instalación y arranque

```
cp .env.example .env    # completar credenciales reales
npm install
npm run seed:places     # opcional: lugares de ejemplo en Xicotepec
npm run migrate:entity-fields # una sola vez
npm run dev             # desarrollo, con nodemon
# o en producción:
npm start
```

Nota: Si @tensorflow/tfjs-node no logra compilar su binario nativo (típico en Windows sin Visual Studio + node-gyp), la instalación NO falla: el servicio de IA cae automáticamente a @tensorflow/tfjs (puro JavaScript, sin binarios). El log de arranque indica cuál de los dos quedó activo.

Una vez arriba, el servidor expone la API bajo el prefijo /api/v1, por ejemplo:

```
GET http://<host>:<puerto>/api/v1/health
```

3.3 Endpoints que consume el reloj (Módulo Wearable)

Endpoint	Uso desde el reloj
POST /api/v1/wearable/pair/generate-code	Llamado desde el teléfono (autenticado): genera el código de 6 dígitos para vincular un reloj nuevo
POST /api/v1/wearable/pair/redeem	Llamado desde el reloj (sin sesión previa): canjea el código por credenciales propias del dispositivo
POST /api/v1/wearable/auth/refresh	Llamado desde el reloj: renueva la sesión usando el refreshToken guardado localmente
GET /api/v1/wearable/itinerary/active	Itinerario activo, payload reducido (título, destino, actividades de hoy)
GET /api/v1/wearable/nearby?lat=&lng=	Lugares cercanos, payload mínimo para el mini-mapa del reloj
POST /api/v1/wearable/location	Ping de ubicación en tiempo real; puede disparar alertas de proximidad
GET /api/v1/wearable/alerts?lat=&lng=	Igual a /location pero sin registrar el evento (refresco de notificación)

4. Configuración de la app Wear OS para apuntar al backend

La URL base del backend está definida como constante de compilación (BuildConfig.API_BASE_URL) en app/build.gradle.kts, con un valor distinto por tipo de build:

Build type	API_BASE_URL
release	https://api.travexperience.com/api/v1/ (dominio de producción)
debug	http://<IP-LOCAL>:4000/api/v1/ (IP de tu red local, para probar con el reloj físico o el emulador apuntando a tu backend en desarrollo)

Para apuntar el build debug a tu propio backend local, edita el valor en app/build.gradle.kts:

```
buildTypes {
    debug {
        buildConfigField("String", "API_BASE_URL", "\"http://TU-IP-LOCAL:4000/api/v1/\"")
    }
}
```

Nota: La IP debe ser accesible desde el reloj: si pruebas con un reloj físico conectado por WiFi, usa la IP de tu computadora en esa misma red (ej. 192.168.x.x), no 127.0.0.1 / localhost.

El proyecto ya incluye android:usesCleartextTraffic="true" y un network_security_config.xml que permite tráfico HTTP sin cifrar — necesario solo para desarrollo contra IPs locales. En producción, el backend debe servirse por HTTPS.

5. Instalación de la app en el reloj

5.1 Desde Android Studio (desarrollo)

1. Con el emulador de Wear OS corriendo, o el reloj físico conectado por ADB sobre WiFi, selecciona el dispositivo en la barra de Android Studio.
2. Presiona Run  (o Shift+F10). Esto compila, instala y abre la app automáticamente.

5.2 Instalación manual de un APK (release o debug ya compilado)

```
adb connect <IP-DEL-RELOJ>:5555 # solo si es por WiFi
adb install -r app/build/outputs/apk/release/app-release.apk
```

Nota: Ver el Manual de Build para los comandos completos de compilación (gradlew assembleDebug / assembleRelease) y firma del APK de release.

6. Verificación del flujo completo (checklist post-deploy)

- El backend responde en GET /api/v1/health.
- El backend tiene conexión activa a MySQL y MongoDB (revisar logs de arranque).
- La app móvil o web puede iniciar sesión y generar un código en /wearable/pair/generate-code.
- La app del reloj está instalada y configurada con el API_BASE_URL correcto para el entorno (local o producción).
- El reloj logra canjear el código (pantalla “Vincular” → pasa a la pantalla de Itinerario).
- El reloj muestra el itinerario activo y el clima correctamente.
- La pantalla “Cerca de Mí” muestra lugares tras otorgar el permiso de ubicación.
- Desvincular desde la pantalla Cuenta cierra la sesión correctamente y regresa a la pantalla de vinculación.